

Speaker Notes

Contents

- [1 1. Containerization](#)
- [2 2. Two commands, one flag apart](#)
- [3 3. Roadmap](#)
- [4 4. Takeaways](#)
- [5 5. In scope, and out of scope](#)
- [6 6. The need](#)
- [7 7. One machine, many workloads](#)
- [8 8. One machine, two applications with different needs](#)
- [9 9. A virtual machine solves both, and what it costs](#)
- [10 10. A third answer, between the two](#)
- [11 11. What a container is](#)
- [12 12. Run your first container](#)
- [13 13. The engine is the program you install](#)
- [14 14. Linux: install the engine directly](#)
- [15 15. macOS first: Homebrew installs the command-line tools](#)
- [16 16. macOS: a Linux virtual machine, driven from the shell](#)
- [17 17. Windows: WSL 2, and two settings to check](#)
- [18 18. macOS and Windows needed a virtual machine](#)
- [19 19. One shell, and the same commands after it](#)
- [20 20. An image is a stack of layers](#)
- [21 21. An image is not a container](#)
- [22 22. A registry is where images come from](#)
- [23 23. Predict: where does the container appear?](#)
- [24 24. Observe: the same question, three platforms](#)
- [25 25. Explain: the engine started a process](#)
- [26 26. Predict: what survives stop, start and remove?](#)
- [27 27. Observe: it survives one of the two](#)
- [28 28. The writable layer, and the image below it](#)
- [29 29. A bind mount shows a host directory](#)
- [30 30. The pitfall: who owns the files](#)
- [31 31. Why a bind mount is not enough](#)
- [32 32. A volume outlives the container](#)
- [33 33. Observe: one write, three destinations](#)

34 34. The problem: a run you cannot repeat

35 35. The application: a page that counts its visits

36 36. A minimal Dockerfile

37 37. Observe: one row for each instruction

38 38. One file brings the same setup up

39 39. Predict: how much memory will it report?

40 40. Observe: the limit holds, the report does not

41 41. Predict: two tracing questions, not one

42 42. Observe: one works, one refuses

43 43. Observe: what the option switched on

44 44. Five questions the commands left open

45 45. What is underneath

46 46. Nothing was hidden from the host

47 47. A namespace is a kernel object, not a file

48 48. One idea, applied eight times

49 49. Three system calls, all traceable

50 50. The user namespace is the load-bearing one

51 51. Union filesystem: what it sees as root

52 52. Layering is copy-on-write, one instruction at a time

53 53. The unit of sharing is a whole file

54 54. The mount namespace, and what a volume really is

55 55. Control groups: what it can consume

56 56. Namespaces hide, control groups constrain

57 57. Nobody gave `/proc/meminfo` a namespace

58 58. Capabilities: how much of root is left

59 59. Read the default set from the engine's own source

60 60. Seccomp: which calls it may issue

61 61. The refusal named a mechanism

62 62. One file holds all five requests

63 63. A container is a list of requests

64 64. Two architectures

65 65. Engine, OCI runtime, and the work

66 66. Docker: a client, a daemon, and two tiers

67 67. What the host process table actually shows

68 68. What the daemon actually controls

69 69. Podman: fork and exec, with a monitor left behind

70 70. What a start without root actually does

71 71. Predict: whose trace shows the namespace call?

72 72. Observe: only one of the two traces holds the call

73 73. Explain: the call is the same, the parent is not

74 74. Two OCI runtimes, one specification

75 75. Rootless: two different concessions

76 76. Host policy overrides every engine option

77 77. The network path without root is a userspace stack

78 78. One image, either engine

79 79. The disagreement is about ownership

80 80. What follows from that

81 81. Same verbs, different engine

82 82. Why macOS and Windows need a Linux virtual machine

83 83. Where "a Linux virtual machine" stops being the answer

84 84. One shared machine, or one for each container

85 85. Where it breaks

86 86. The boundary is the system-call interface

87 87. A container escape, in four steps

88 88. One class, disclosed four times

89 89. When the kernel is the bug

90 90. One systematic study, and what it counted

91 91. The other boundary is narrower, not absolute

92 92. Containers went back inside virtual machines

93 93. Which containers are not virtual machines

94 94. The leak you can measure: files across the boundary

95 95. The leak you can trip over: the wrong architecture

96 96. What is inside an image nobody opened

97 97. Three things the evidence leaves open

98 98. Where it came from

99 99. One line, 1982 to 2015

100 100. Three lanes that met

101 101. What 2013 added, and what it did not

102 102. Kubernetes dropped Docker, and the images kept working

103 103. Takeaways

104 104. What follows

105 105. References

Generated by the agentic vault — an autonomous research and authoring harness. Every factual claim was gated against a cited source registry before this document was produced.

v3 · dff05f3a8 · built 2026-09-30

Slides: 105

11. Containerization

On screen:

PB158

A container is not a lightweight virtual machine. It is an ordinary process that Linux kernel mechanisms isolate.

Ondřej (Ondra) Krajíček me@ondrejkrasicek.com · www.linkedin.com/in/OndrejKrajicek

Built at render time

- A container is an ordinary process. Unrelated kernel interfaces restrict what it sees and what it consumes [V1].
- A container is "not a virtual machine at all, it's just processes running in the same Linux kernel" [S18].
- The shared kernel makes the isolation weaker than a virtual machine's separation [V7].
- The engine's own teaching documentation says the same thing: containers are isolated processes [S75].

2 2. Two commands, one flag apart

On screen:

Part I · The need

One flag is the only difference between these two commands. Only the second one works.

Docker

[code]

Podman

[code]

- *Without the flag, `strace` is refused inside the container. With the flag it works.*
- *Both engines answer the same way. You run both commands in Part II.*

- Under the default setting, `strace` attaches only with `CAP_SYS_PTRACE`, or when the target is a descendant of the tracer [S67].
- Neither engine grants `CAP_SYS_PTRACE` by default [S42] [S43].
- The refusal is the capability machinery working as documented [V10].
- A container can be taken apart with `ls`, `cat` and `strace` [V12].
- The image in this example is built in Part II, step 4 [inference].

Collect the room's guesses.

3 3. Roadmap

On screen:

Seven parts. You run containers in Part II and see how they work in Part III.

I · The need

Three problems, all older than the word.

II · Run your first container

Install an engine, run it, keep the data, repeat the run.

III · What is underneath

Five kernel mechanisms, five answers.

IV · Two architectures

The same mechanisms, assembled differently.

V · What follows

Why macOS and Windows need Linux inside.

VI · Where it breaks

The boundary, the escapes, the leaks.

VII · Where it came from

Optional. Timelines.

The room runs the hands-on work on its own laptops in Part II. Say why the order is this way: each kernel mechanism in Part III answers a question the room already met when it ran the commands in Part II. Part VII is optional. Skip it without loss.

4 4. Takeaways

On screen:

1

No kernel object

Unrelated kernel interfaces restrict an ordinary process.

2

Not a small machine

A container shares the host kernel. A virtual machine runs its own.

3

Same parts, two shapes

Same mechanisms, different owner of the container process.

4

Linux, or a Linux inside

A Linux container on macOS or Windows runs inside a Linux virtual machine.

5

Configuration, not magic

What it may do is written in settings you can read and change.

- A container is not a kernel object [T1] [V1].
- A container is not a lightweight virtual machine [T2] [V7].
- A container does not boot another copy of the operating system, and a virtual machine does [S63].
- Docker and Podman differ in architecture, not in kernel usage [T3] [V15].
- A Linux container on macOS or Windows runs inside a Linux virtual machine [T4] [V9].
- What a process may do is readable configuration [T5] [V10].

5 5. In scope, and out of scope

On screen:

Covered

The five kernel mechanisms, each observed with a command.

Running one engine on your own laptop, from a first run to a repeatable setup.

Docker against Podman, by architecture.

Where the abstraction leaks, and the escapes recorded through it.

Not covered

Orchestration across several machines.

Container networking beyond one published port.

Image hardening, image size and the build cache.

Supply-chain practice.

- A student observes each of the five kernel mechanisms with a command.
- One engine runs from an image to a repeatable setup.
- Image bloat and supply-chain risk get one mention each, and no exercise.
- A one-container Compose file is in scope. Orchestration across machines is not.

6 6. The need

On screen:

Part I

Three problems, all older than the word container.

Part I states the problems. No mechanism yet.

7 7. One machine, many workloads

On screen:

*Part I · The need**One server runs several workloads, and three conditions must hold at once.*

- *No application reads another application's data.*
- *No application changes how another one runs.*
- *No application takes so much of the machine that the others stop.*

*Partial answer**An old call named `chroot` shows a process one subtree of the file tree and hides the rest.**Why it fails**A process inside `chroot` still lists the other processes and reaches the network.**Isolation for workloads that cooperate, and one defence among several. Every container uses the host kernel, so it is not a wall.*

- The Solaris Zones designers state the need as isolation plus resource control [S58].
- `chroot` does not compartmentalise the process or networking spaces [S19].
- A process inside `chroot` therefore still reaches the other processes and the network [S19] [inference].
- The author of FreeBSD jails states that `chroot` was never intended as a security mechanism [S20].
- The shared kernel makes the isolation weaker than a virtual machine's separation [V7].
- Kubernetes states that containers "offer a weaker isolation boundary than virtual machines that utilize hardware-based virtualization" [S73].
- Part VI shows where that boundary has failed [inference].

8 8. One machine, two applications with different needs

On screen:*Part I · The need**Two applications on one host need different versions of the same thing.*

- *The Solaris Zones paper records two applications at one customer site, both hard-coded to listen on port 80. The customer bought two servers.*

- *Hard-coded paths and log locations can make two versions of one application hard to install side by side.*
- *Boettiger cites a University of Arizona study: less than half of the software published with computer-systems papers could be built or installed.*

The need is a package that carries the whole user-space environment with it.

- The paper records "two poorly written applications" contending for port 80, resolved "by buying two servers" [S58].
- The same paper names hard-coded pathnames as a barrier, including the place a log file is written [S58].
- They can make two versions of one application difficult to deploy on one node [S58].
- Boettiger reports a University of Arizona study finding that "less than 50% of software could even be successfully built or installed" [S62].
- The literature calls this barrier "Dependency Hell" [S62].
- The need is packaging, and it is separate from the isolation need [inference].

9 9. A virtual machine solves both, and what it costs

On screen:

Part I · The need

A virtual machine is a simulated computer that runs its own kernel, so the guest kernel is not the host kernel.

11 s

to start a virtual machine, in a measurement Felter's paper cites

under 1 s

to start a container, on the same hardware

1

kernel per virtual machine, booted and held in memory

- *Felter's own measurements found containers equal to or better than virtual machines on almost every workload, and both needed tuning for heavy input and output.*

• *Speed is not the argument. Start-up time and density are.*

- Felter's paper reports that "other measurements have shown" under one second against eleven seconds on that hardware [S63].
- The reason given is that a container does not boot another copy of the operating system [S63].
- Felter's own results report containers "equal or better" in almost all cases, and add that both need tuning for input-heavy work [S63].
- The Solaris Zones paper names administering many operating system instances as the other cost [S58].

10 10. A third answer, between the two

On screen:

Part I · The need

A microVM is a virtual machine with almost every simulated device removed. It keeps its own kernel and gives up the rest.

	CONTAINER	MICROVM	FULL VIRTUAL MACHINE
<i>Kernel</i>	<i>the host's</i>	<i>its own</i>	<i>its own</i>
<i>Boundary</i>	<i>the system-call interface</i>	<i>hardware, plus the monitor</i>	<i>hardware, plus the monitor</i>
<i>Start</i>	<i>fastest</i>	<i>fast</i>	<i>slowest</i>

- *Firecracker is one microVM monitor, and it runs a minimal guest kernel of its own.*
- *Its authors report a boot to application code in under 125 ms, and under 5 MB of added memory for each container.*
- *AWS runs Lambda and Fargate on it, and states it built Firecracker rather than choose between isolation and compatibility.*

- A microVM gives each sandbox "its own virtual hardware, page tables, and operating system kernel" [S38].

- The paper's claim is that this "moves the security-critical interface from the OS boundary to a boundary supported in hardware and comparatively simpler software", which is the monitor [S38].
- With its minimal guest-kernel configuration Firecracker "boots to application code in less than 125ms" and "offers memory overhead of less than 5MB per container" [S38].
- AWS runs Firecracker in Lambda and Fargate, "where it supports millions of production workloads" [S38].
- Reliance on a single kernel means "a fundamental tradeoff between security and code compatibility" [S38].
- AWS states "We built Firecracker because we didn't want to choose" [S38].
- A container does not boot another copy of the operating system, so it starts fastest of the three [S63].

11 11. What a container is

On screen:

Part I · The need

A container is a process, or a tree of processes, that runs with its own view of the machine on the host's kernel.

Its own view

Its own files, its own process list, its own network.

Its own share

A set amount of memory and of processor time.

The host's kernel

No second kernel boots. It starts as fast as an ordinary program.

- *This course's own Linux toolchain is an image. You start a container from it in Part II.*

- A container is an ordinary process. Unrelated kernel interfaces restrict what it sees and what it consumes [V1].
- The engine's own teaching documentation describes containers as isolated processes [S75].

- A container does not boot another copy of the operating system, and a virtual machine does [S63].
- This course's own Linux toolchain is a Debian-family image, which the room has not run before this deck [V17].
- No mechanism name is given yet. Part III names them [inference].

12 12. Run your first container

On screen:

Part II

Install an engine, run a container, keep its data, and make the run repeatable.

Part II uses the tool. It explains nothing about the kernel yet. Every step closes with one question that Part III answers.

13 13. The engine is the program you install

On screen:

Part II · Setting up

A container engine takes a command, finds the files the application needs, and starts the container.

1

Resolve

Turn the name `python:3.12-slim` into a list of files to fetch from a server.

2

Unpack

Fetch the missing files and unpack them.

3

Configure

Write a configuration naming what the process may see and use.

4

*Start**Start the process with that configuration.*

- *Docker and Podman are container engines, and both take the same verbs from the command line.*

- Docker's engine is a client and a daemon that builds, runs and distributes containers [S10].
- Podman is "a fully featured container engine that is a simple daemonless tool" [S13].
- Docker's licence lists education among the uses for which Docker Desktop is free [S50].
- The command line is enough for every step in this topic [inference].
- Step 3 writes the file the OCI runtime specification defines [S7].

14 14. Linux: install the engine directly

On screen:

Part II · Setting up

Linux runs containers natively. Install one engine, and follow it for the rest of the topic.

Docker

[code]

Podman

[code]

If Podman refuses

Podman 6 needs version 2 of the kernel's resource-limit interface. `stat -fc %T /sys/fs/cgroup` prints `cgroup2fs` for version 2.

Version

A distribution package can be older than Podman 6. Read `podman --version`.

- Docker publishes installation steps for each Linux distribution, not one script [S51].

- Docker's instructions start by removing the distribution's `docker.io` and related packages, "which may conflict with the official packages provided by Docker" [S82].
- Docker publishes the same install steps as a convenience script fetched from `get.docker.com` [S82].
- Docker documents `sudo usermod -aG docker $USER` and `newgrp docker` as the step that lets a non-root account use the daemon socket [S85].
- A distribution package can lag the upstream Podman release, so a version-specific claim needs `podman --version` first [inference].
- Podman publishes package steps for each major Linux distribution [S52]. It is built in on Fedora CoreOS and Fedora Silverblue [S52].
- Podman 6.0.0 removed "support for running on cgroups v1 systems" [S65].
- Part III explains control groups [inference].

15 15. macOS first: Homebrew installs the command-line tools

On screen:

Part II · Setting up

Homebrew is a package manager for macOS. It installs command-line programs, as `apt` does on Ubuntu.

[code]

1 · Run the installer

One command, taken from `brew.sh`. It asks for your password once.

2 · Next steps

The installer prints lines under "Next steps". Run them. They put `brew` on your `PATH`.

3 · Where it installs

`/opt/homebrew` on Apple Silicon, `/usr/local` on an Intel Mac. The `PATH` lines differ for that reason.

- *Check the installation with `brew --version`. Only then run `brew install lima`.*

- Homebrew "installs the command-line tools and applications you need across macOS, Linux and WSL" [S83].

- The one-line installer is one of the install routes the project publishes [S83].
- Homebrew installs under "/opt/homebrew on Apple Silicon" and "/usr/local on macOS Intel" [S83].
- The installer prints directions for updating the shell's configuration, and Homebrew "will not work" until they are followed [S86].

16 16. macOS: a Linux virtual machine, driven from the shell

On screen:

Part II · Setting up

macOS has no Linux kernel. The engine runs inside a Linux virtual machine that Lima creates.

Docker

[code]

Podman

[code]

Template syntax

Current Lima takes `template:docker` , not the older `template://docker` .

Read-only home

Lima mounts your home directory read-only. A write there fails by design.

Intel Mac

Podman 6 dropped Intel Macs. Use the Lima `docker` template there.

- Lima documents the template-URL change from the 1.x form [S53].
- The Lima defaults are four cores, 4 GiB of memory and a read-only home mount [S54].
- `podman machine` instead mounts the home directory read-write by default [S55].
- Podman 6.0.0 removed "support for running on Intel Macs" [S65].
- Why a Linux virtual machine is needed at all is Part V [inference].

17 17. Windows: WSL 2, and two settings to check

On screen:

Part II · Setting up

Windows Subsystem for Linux runs a real Linux kernel in a small virtual machine.

Docker

[code]

Podman

[code]

Virtualisation

Error 0x80370102: turn on Virtual Machine Platform, then BIOS virtualisation.

Systemd, for Podman

Podman without root needs the Linux service manager. Set `systemd=true` under `[boot]` , then run `wsl --shutdown` .

- WSL 2 uses "a Linux kernel inside of a lightweight utility virtual machine" [S46].
- The WSL virtual machine's memory default is 50 percent of host memory [S71].
- Podman without root needs systemd, and WSL 2 does not enable it by default [V38].
- Systemd is now the default for the Ubuntu that `wsl --install` installs, so check it with `systemctl status` first [S47].
- The `[boot]` edit needs WSL version 0.67.6 or newer [S47].
- Podman 6.0.0 removed control groups version 1 support, so `v1` here is a hard stop [S65].
- For error 0x80370102 Microsoft asks the reader to enable the Virtual Machine Platform feature and to "ensure virtualization is enabled in the BIOS" [S79].

18 18. macOS and Windows needed a virtual machine

On screen:

Part II · Setting up

A container on macOS or Windows needs a Linux virtual machine, and that machine costs a boot only once.

Why

The mechanisms are Linux features

The kernel features a container needs exist in the Linux kernel.

The macOS and Windows kernels do not have them.

So Lima and WSL 2 supply a Linux kernel, inside a virtual machine.

What it buys back

One machine, many containers

The virtual machine boots once.

Each container after that costs one process start, not one boot.

- Every Docker on a Mac is Docker inside a Linux virtual machine [V9].
- Lima creates that Linux virtual machine on macOS [S53].
- WSL 2 supplies the Linux kernel on Windows in the same way [S46].
- The cost is paid once for the virtual machine, not once per container [inference].
- Part V gives the mechanism and the exception [inference].

19 19. One shell, and the same commands after it

On screen:

Part II · Setting up

Every command from here runs in a Linux shell. Open that shell once, and stay in it.

Linux

Your own terminal. Nothing to open.

macOS

`limactl shell docker`, or `limactl shell podman`, opens a shell inside the virtual machine.

Windows

`wsl` opens a shell inside the Linux distribution.

- Make your working files in that shell. A program reads a file on the macOS or Windows side more slowly than one inside the Linux filesystem, and Part VI gives the measurement.
- On Windows the subsystem supplies the ownership of files under `/mnt/c`.

- Lima runs the engine inside a virtual machine, reached with `limactl shell` [S53].
- WSL 2 runs a Linux kernel in a small virtual machine, reached with `wsl` [S46].
- Podman keeps a deliberately compatible command set, so most commands differ by one word [S13].
- Microsoft advises keeping files on the Linux drive for speed [S72].
- WSL adds no metadata to Windows files by default, so Linux permissions under `/mnt/c` are supplied rather than stored [S71].

20 20. An image is a stack of layers

On screen:

Part II · Step 1 · An image and a container

An image is the set of files an application runs in. Those files are stored as layers.

1

Layer

One set of file changes: files added, replaced or removed.

2

Stack

A file in a higher layer hides the file of the same name below it.

3

Manifest

The list that names each layer by a hash of its content.

- Every layer is read-only. The files in an image never change after the build.

- The image specification defines a manifest naming content-addressed layer archives [S8].

- Each layer is one set of filesystem changes [S23].
- The image's layers are read-only, and a container adds its own writable layer above them [S64].
- A file in a higher layer hides the file of the same name in a lower one [S6].

21 21. An image is not a container

On screen:

Part II · Step 1 · An image and a container

Two words, and every later sentence depends on telling them apart.

Image

Inert data

A stack of read-only layers, plus the manifest that names them.

Nothing about it runs. It can sit in a file or on a server.

Container

A running process

A process, or a tree of processes, started from an image.

It has a process identifier on the host.

It ends when its command ends.

You cannot download a container. You download an image and start one.

- The image specification defines a manifest naming content-addressed layer archives [S8].
- A container is a process or process tree started from those layers [V1].
- The image holds no writable layer, so a running container does not change it [inference].
- One image starts many containers, and each one gets its own writable layer [S64] [inference].

22 22. A registry is where images come from

On screen:

Part II · Step 1 · An image and a container

A registry stores images under a name and a hash, and serves them over a documented protocol.

Docker

[code]

Podman

[code]

- *The engine turns the name into a manifest, then fetches the layers the manifest names.*
- *The name has two halves. `python` is the repository, and `3.12-slim` is the tag.*
- *Layers already on disk are not fetched again.*
- *Docker Hub allows an unauthenticated address 100 pulls in six hours, and a room behind one shared address shares that budget.*
- *Pull the images before you need them. Pull 101 returns an error that looks like broken Wi-Fi.*

- Docker Hub meters "Unauthenticated users | 100 per IPv4 address or IPv6 /64 sub-net" per six hours [S49].
- The manifest names the layers by the hash of their content [S8].
- Many machines behind one shared address draw on one budget [S49] [inference].

Pre-pull before the session.

23 23. Predict: where does the container appear?

On screen:

Part II · Step 1 · An image and a container

Write your answer down before anything runs.

Docker

[code]

Podman

[code]

*Question 1**Does the web server appear in the host process list? Does it survive closing the terminal?**Question 2**Run `ps` inside the container. How many processes does it see?*

- `--rm` deletes the container when it stops. `-d` returns your shell at once.
- `--name` labels it. `-p 8080:80` maps host port 8080 to port 80 inside.

- `--rm` removes the container when it exits, `-d` detaches it, and `-p` publishes a container port on a host port [S61].
- A written prediction before the observation gives the room a stake in the result [S57].
- The container is a process, or a tree of processes, on the host [V1].
- In a PID namespace the first process becomes PID 1, and the others disappear from view [S18].

Collect the answers first.

24 24. Observe: the same question, three platforms

On screen:*Part II · Step 1 · An image and a container**Linux**host shell**The web server is in the host process list.**`ps -ef` on your own machine**macOS**Lima shell**The host list shows one virtual machine. The server is inside it.**`limactl shell` then `ps -ef`**Windows**WSL shell**The same shape as macOS. Run the command inside the Linux shell.*

```
wsl then ps -ef
```

- *Inside the container, ps shows one or two processes and nothing else. On the host the same processes are there, under different numbers.*

- The container process is visible on the Linux host running the engine [V1].
- On macOS and Windows the host is the Linux virtual machine, not the laptop [V9].
- A process in a PID namespace sees itself as PID 1 and cannot see the rest [S18].
- Parentage differs by engine, and Part IV takes that apart [inference].

25 25. Explain: the engine started a process

On screen:

Part II · Step 1 · An image and a container

The engine created nothing new. It started a process with an unusual view of the machine.

On the host

An ordinary process

It has a process identifier.

It can be listed, signalled and traced.

The engine started it.

Inside

A very short process list

The command is process number 1.

Every other process on the machine is missing.

The hostname is different too.

Open question 1: one process, two process lists. What makes the second one short?

- The container is an ordinary process. Unrelated kernel interfaces restrict what it sees and what it consumes [V1].
- In a PID namespace the process becomes PID 1 and the others are gone [S18].
- The answer is the PID namespace. Part III teaches it [inference].

26 26. Predict: what survives stop, start and remove?

On screen:

Part II · Step 2 · Storing data

The application writes a file. Write down what happens to that file.

Docker

[code]

Podman

[code]

Three questions

Is /data.txt there after stop and start? Is it there after remove and a new run? Did the image change?

- A writable layer holds the files created inside a container [S64].
- Data on that layer "doesn't persist when the container is destroyed" [S64].
- A written prediction first is the method this course uses [S57].

27 27. Observe: it survives one of the two

On screen:

Part II · Step 2 · Storing data

kept

after stop and start, same container

gone

after remove, in a new container

0

changes to the image, either way

- *Stopping a container does not delete it. Removing it does.*

- *The new container started from the same image, and the image never held the file.*

- The writable layer holds files created inside the container [S64].
- That layer does not persist when the container is destroyed [S64].
- An image is a stack of read-only layers, so a running container does not modify it [S8] [inference].

28 28. The writable layer, and the image below it

On screen:

Part II · Step 2 · Storing data

An image is a stack of read-only layers. Starting a container puts one writable layer on top of that stack.

[code]

- *The topmost layer that holds the file answers a read.*
- *A write always lands in the writable layer.*
- *Remove the container and the writable layer goes with it. The image is untouched.*

Open question 2: layer is the tool's word. Which kernel mechanism is it?

- The writable layer holds every file created inside a container [S64].
- An image is a stack of read-only layers named by the hash of their content [S8].
- Each layer is one set of filesystem changes [S23].
- Open question 2: a layer is a word from the tool. The kernel mechanism is in Part III [inference].

29 29. A bind mount shows a host directory

On screen:

Part II · Step 3 · Keeping the data

A bind mount makes a directory that already exists on the host appear at a path inside the container. To mount a directory is to make its contents appear at a chosen path.

*[code]**Docker**[code]**Podman**[code]*

- A volume is engine-managed storage, and a bind mount is an existing host directory [S64].
- The Lima default mounts the macOS home directory read-only, so a bind mount under `~` refuses the write and `/tmp` does not [S54] [inference].
- On Windows, keep the directory inside the Linux filesystem rather than under `/mnt/c` [S72].

30 30. The pitfall: who owns the files

On screen:

Part II · Step 3 · Keeping the data

A bind mount can leave files under an owner nobody chose.

1 · What you see

`ls -l /tmp/work` shows `new.txt` under an unexpected owner. Under Docker's root daemon it is root.

2 · Why

Whichever account writes the file decides its owner outside. Rootless Podman maps it into the range reserved in `/etc/subuid`.

3 · Fix

Run the container under your own number with `--user $(id -u)`, or add Podman's `:U` suffix to `-v`.

4 · Windows

Under `/mnt/c`, WSL supplies the ownership.

- The documented rule is to treat root within a container as root outside of it [S32]. Under Docker's root daemon a container therefore writes host files as root [inference].
- Docker's rootless mode runs the daemon and the containers inside a user namespace [S11].
- It needs at least 65,536 subordinate identifiers for the user [S11].
- Rootless Podman reads `/etc/subuid` and `/etc/subgid` and builds the identifier mapping itself [V32].
- A file written by a non-root user inside a rootless container therefore carries a number from the reserved range [inference].
- WSL sets file metadata to `disabled` by default, so permissions under `/mnt/c` are supplied rather than stored [S71].

31 31. Why a bind mount is not enough

On screen:

Part II · Step 3 · Keeping the data

A bind mount works. It also ties the container to one machine.

1 · One machine

The command needs this directory at this path. Another laptop does not have it.

2 · Your real files

The container can change or delete your files on the host.

3 · Across a virtual machine

On macOS and Windows the files cross into the Linux virtual machine. That is slow.

4 · Owners

Files can get an owner nobody chose.

Missing: storage the engine owns, under a name, on any machine.

- Bind mounts "rely on the host machine's filesystem having a specific directory structure available" [S80].

- A process in the container can create, modify or delete files on the host through a bind mount [S80].
- One measurement on one macOS codebase found bind-mounted access to many small files about 3.5 times slower than Linux [S44].
- Moving that directory into a named volume brought the timings back in line with Linux [S44].
- The Lima default shares the home directory read-only [S54].
- On Windows, files under `/mnt/c` are slower than files inside the Linux filesystem [S72].
- The ownership problem is the previous slide [inference].

32 32. A volume outlives the container

On screen:

Part II · Step 3 · Keeping the data

A volume is storage the engine creates and tracks under a name. Its lifetime is not the container's lifetime, and you name no host path.

Docker

[code]

Podman

[code]

- *The first container is removed and the file is still there.*
- *A second container can mount the same volume and read it.*
- *A volume stays until you delete it.*
- *The same command works on any machine, because you give the volume a name and not a host path.*

- Volumes "retain data even after the containers using them are removed" [S64].
- The engine's own wording is that volumes are "completely managed by Docker" [S76].
- Docker calls volumes "the preferred mechanism for persisting data" [S76].
- A named volume needs no directory on the host, so the command does not depend on one machine's layout [S76] [inference].

33 33. Observe: one write, three destinations

On screen:

Part II · Step 3 · Keeping the data

Writable layer

no option

Survives stop and start. Gone after remove.

Default for every path

Volume

-v name:/path

Survives remove. Readable by the next container.

Engine creates and tracks it

Bind mount

-v /host:/path

Survives remove. Visible with `ls` in the shell you started in.

Host directory you name

Same write, three lifetimes. Open question 3: how does one directory appear inside one process and not the others?

- The writable layer does not persist past the container [S64].
- A volume retains data after the container using it is removed [S64].
- A bind mount reads and writes the host filesystem directly [S64].
- Contrasting the three side by side is what prepares the room for the mechanism [S57].
- The answer to the open question is the mount namespace, in Part III [inference].

34 34. The problem: a run you cannot repeat

On screen:

Part II · Step 4 · Making it repeatable

You typed everything so far. Nothing on your machine records what you typed.

- *You pulled the image by hand.*
- *You created the volume by hand.*
- *You chose the port by hand.*
- *A second person must repeat every command exactly, and so must you next week.*
- *Two files fix this. One builds the image. One records the run.*

Dockerfile

A recipe that builds an image from a base image and some commands.

Compose file

A record of one run: which image, which volume, which port.

- An image is built from a base image plus instructions, one layer per instruction [S23].
- The image configuration's `history` array records the command that created each layer [S60].
- Making one container repeatable is the goal, and orchestration is out of scope [inference].

35 35. The application: a page that counts its visits

On screen:

Part II · Step 4 · Making it repeatable

Save this as `app.py` in an empty directory. It is also in the deck's `demo/` folder.

[code]

- *With a volume on `/data`, the count survives removing the container.*

- The program uses only the Python standard library, so the base image needs nothing else to run it [inference].
- The count lives in `/data/count.txt`, the same path the volume and the Compose file mount, which ties step 3 to step 4 [inference].
- The file ships in the talk's `demo/` bundle, so a student can download it rather than copy it from a slide [inference].

36 36. A minimal Dockerfile

On screen:

Part II · Step 4 · Making it repeatable

Save this as `Dockerfile` next to `app.py`. Each instruction that changes the filesystem adds one layer.

[code]

Docker

[code]

Podman

[code]

- Image layers form a stack, each adding content on top of the ones before it [S23].
- The image configuration marks an instruction that created no layer with `empty_layer` [S60].
- The build cache is not taught here [inference].

37 37. Observe: one row for each instruction

On screen:

Part II · Step 4 · Making it repeatable

Docker

[code]

Podman

[code]

[code]

- The rows read bottom to top, and the base image is at the bottom.
- The `CMD` row is 0 bytes. It changed no file, so it added no layer.

- The `history` field "Describes the history of each layer", and `created_by` is the command that created it [S60].

- A history entry with no filesystem change carries `empty_layer` [S60].
- The rows read bottom to top, and the base image sits at the bottom [inference].

38 38. One file brings the same setup up

On screen:

Part II · Step 4 · Making it repeatable

[code]

Docker

[code]

One command builds the image, creates the volume and publishes the port.

Podman

[code]

Without `podman-compose` the command reports no provider.

- *Compose is the one place where the engines differ in substance.*
- *It names its volume after the project, so the count starts again.*

- "podman compose is a thin wrapper around an external compose provider such as docker-compose or podman-compose" [S66].
- Docker ships the compose implementation with the engine, and Podman ships only the wrapper [S66] [inference].
- The file records one container's image, volume and port [inference].
- Compose names its volume after the project, so the run does not reuse the `appdata` volume [inference].
- Several services in one file is orchestration and is out of scope [inference].

39 39. Predict: how much memory will it report?

On screen:

Part II · Step 5 · What it may use and do

You start the container with a hard memory limit. Write down what the tool inside will print.

Docker

[code]

Podman

[code]

Question 1

Does `free -h` inside print 256 MB, or the whole machine? Does `nproc` count the host processors?

Question 2

Does the limit actually hold if the program allocates 512 MB?

- *Keep the figure small. On Windows the virtual machine already took half of the memory.*

- `--memory` sets a memory limit on the container [S61], and a control group is what enforces it [S2].
- Docker states that `free` inside a container reports "the host's available swap" [S68].
- `/proc/meminfo` and its neighbours "will always display memory numbers from the host system" [S41].
- The WSL virtual machine defaults to 50 percent of Windows memory [S71].

40 40. Observe: the limit holds, the report does not

On screen:

Part II · Step 5 · What it may use and do

The limit

Enforced

Allocate 512 MB under a 256 MB limit, and the kernel kills the process.

The kernel enforces the limit. The engine does not.

The report

The host total

`free -h` prints the whole machine's memory, and so does `/proc/meminfo`.

`nproc` counts the host processors. The memory limit does not change that number.

Open question 4: the limit is real and the report ignores it. Two different mechanisms are at work.

- `--memory` sets the limit the container runs under [S61], and a control group is what holds it [S2].
- An allocation above that limit fails inside the kernel, so the kernel stops the process and the engine does not [inference].
- Docker states that `free` inside a container reports the host's available swap [S68].
- `/proc/meminfo` and its neighbours "will always display memory numbers from the host system" [S41].
- No namespace covers the processor count [S1] [inference].
- A control group limits and measures consumption, and it does not change what a process reads [S2] [inference].
- So `nproc` inside the container counts the host processors [inference].
- Part III names the two mechanisms and why only one of them hides anything [inference].

41 41. Predict: two tracing questions, not one

On screen:

Part II · Step 5 · What it may use and do

The same tool, two targets. Write down whether each one works.

Docker

[code]

Podman

[code]

- Case A starts its own target.
- Case B attaches to a process that is already running.

- `strace` in case A traces its own child [S67].
- `strace` in case B attaches to a process it did not start [S67].

- The default capability set of both engines excludes `CAP_SYS_PTRACE` [S42] [S43].

42 42. Observe: one works, one refuses

On screen:

Part II · Step 5 · What it may use and do

Case A

own child

It runs, and prints the calls it saw. No extra option was given.

Usually works on a current kernel

Case B

attach

```
strace: attach: ptrace(PTRACE_SEIZE, 1): Operation not permitted
```

Refused

Case B again

--cap-add SYS_PTRACE

Add the option to the `run` above, and the same attach succeeds.

One option changed the answer

[code]

Open question 5: one option, and the refused command was allowed to run. What did it switch on?

- Attaching needs `CAP_SYS_PTRACE`, or a target that is a descendant of the tracer [S67].
- Neither engine grants `CAP_SYS_PTRACE` by default [S42] [S43].
- Adding the option also allowlists the related calls in the filter [S17].
- The refusal is the machinery working as documented, and is not a broken tool [V10].

43 43. Observe: what the option switched on

On screen:

Part II · Step 5 · What it may use and do

A capability is one named piece of root's power, such as the right to trace another process. The kernel lists a process's capabilities in `/proc/self/status`.

Docker

[code]

Podman

[code]

- Run it twice, once with `--cap-add SYS_PTRACE` after `run`.
- Without the option, the `CapEff` line has no `cap_sys_ptrace`. With it, `cap_sys_ptrace` joins the list.
- The two engines start from different lists. Docker keeps `cap_net_raw`, `cap_mknod` and `cap_audit_write`, and Podman drops them.

- A capability is a distinct unit of the privileges traditionally associated with the superuser, which the kernel enables and disables one by one [S3].
- `/proc/<pid>/status` shows a process's capability sets, and `CapEff` is the effective set that the kernel checks [S3] [inference].
- `CAP_SYS_PTRACE` is the capability that allows tracing a process the tracer did not start [S3] [S67].
- Verified 2026-09-29 on kernel 6.8 with Podman 4.9.3 and Docker 29.7.2: the default `CapEff` lacks `cap_sys_ptrace` on both engines, and `--cap-add SYS_PTRACE` adds it [inference].
- Docker's default set holds three capabilities that Podman's does not, `cap_net_raw`, `cap_mknod` and `cap_audit_write` [S42] [S43].

44 44. Five questions the commands left open

On screen:

Part II · Five open questions

1

Two process lists

One process, two views. What makes the second one short?

2

The vanishing file

What is a layer, and why is the image unchanged?

3

The mounted directory

How does a directory appear inside one process only?

4

The honest limit

Why does the limit hold while the report shows the host?

5

The one option

What did the one option change?

Five questions, five kernel mechanisms, in this order.

- The five mechanisms together are what the word container means [V1].

45 45. What is underneath

On screen:

Part III

Part II raised five questions that running the commands could not answer. Part III answers them with five kernel mechanisms, in the same order.

Part III answers Part II. Same application, same commands, new vocabulary.

46 46. Nothing was hidden from the host

On screen:

Part III · Question 1

The container process was in the host list all along. What changed was the list the process itself can read.

Docker

[code]

Podman

[code]

- *The second command prints a host process identifier. The third prints 1, and almost nothing else.*
- *The host numbering did not change. A second numbering was added.*
- *Inside that second numbering the command is process 1, and the host's processes have no number at all.*
- *A process with no number in your table is a process you cannot signal.*

- In a PID namespace the process becomes PID 1 and the other programs are gone [S18].
- A namespace "wraps a global system resource in an abstraction" so the processes inside see their own instance [S1].
- The resource wrapped here is the process identifier space [S1].

47 47. A namespace is a kernel object, not a file

On screen:

Part III · Question 1

A namespace is a kernel object that wraps one global resource, so the processes inside it see their own instance. The entries under `/proc` are handles to it.

[code]

1 · The link

Each entry is a symbolic link whose target names the type and a number.

2 · The handle

Opening one returns a file handle for that namespace, and keeps it alive while open.

3 · The test

Two processes are in the same namespace when the two numbers match.

Compare the numbers for a container process and for your shell.

- Each `/proc/pid/ns/` entry is a symbolic link naming the type and an inode number [S1].
- Opening one "returns a file handle for the corresponding namespace" [S1].
- `/proc` exposes the kernel object and does not represent it as a file [V11].

48 48. One idea, applied eight times

On screen:

Part III · Question 1

Linux defines eight namespace types. Each one wraps one global resource.

TYPE	FLAG	WHAT THE PROCESS SEES ITS OWN COPY OF
PID	CLONE_NEWPID	Process identifiers
Mount	CLONE_NEWNS	Mount points
Network	CLONE_NEWNET	Network devices, addresses and ports
User	CLONE_NEWUSER	User and group identifiers
IPC	CLONE_NEWIPC	Shared memory and message queues
UTS	CLONE_NEWUTS	The hostname
Cgroup	CLONE_NEWCGROUP	Its own position in the ker- nel's resource-limit tree
Time	CLONE_NEWTIME	The boot and monotonic clocks

- The eight types are independent, and UTS is the kernel's old name for the host-name.

- The kernel documents eight namespace types with these flags [S1].
- The IPC namespace partitions System V IPC and POSIX message queues [V18].
- A container takes a private set of namespaces, and the set is a choice [V1].

49 49. Three system calls, all traceable

On screen:

Part III · Question 1

No special interface creates a namespace. Three ordinary system calls do, and `strace` shows them.

`clone(2)`

Creates a process. With one or more `CLONE_NEW*` flags it creates new namespaces at the same time.

`unshare(2)`

Moves the calling process into a new namespace.

`setns(2)`

Joins an existing namespace, named by a file descriptor from `/proc`.

[code]

- `--mount-proc` gives the new namespace its own `/proc`, which is what `ps` reads. No engine ran.
- Part IV traces both engines and names the program that makes the call.

- `clone(2)` creates new namespaces when the flags request them [S1].
- Part IV traces `docker run` and `podman run` and finds the caller [inference].
- `unshare(2)` moves the caller to a new namespace, and `setns(2)` joins one by descriptor [S1].
- A short program that calls `clone` with these flags builds a container without an engine [S56].

50 50. The user namespace is the load-bearing one

On screen:

Part III · Question 1

Only one namespace type changes which account counts as root inside it.

- *The user namespace remaps user and group numbers, so root inside it is an ordinary account outside.*
- *Creating any other namespace type needs administrative privilege an ordinary account does not hold.*
- *Creating a user namespace does not, unless the Linux distribution switches that off. Ubuntu restricts it by default from release 23.10.*
- *That single exception is what containers without root rest on.*

- `CLONE_NEWUSER` appeared in 2.6.23 and the final pieces landed in 3.8 [S21].
- Every namespace type other than the user namespace requires `CAP_SYS_ADMIN` to create [S21].
- The user namespace is the one that isolates user and group identifiers [S1].
- Ubuntu 24.04 "restricts the use of unprivileged user namespaces" by default [S48].
- Rootless containers are a consequence of that single decision [V1].
- Part IV shows what both engines build on top of it [inference].

51 51. Union filesystem: what it sees as root

On screen:

Part III · Question 2

A union filesystem presents several read-only directories plus one writable directory as a single tree.

On Linux this filesystem is `overlayfs`, and it takes three directories as mount options.

lowerdir

The read-only image layers, stacked.

upperdir

The container's writable directory.

merged

The tree the process sees as its root.

- *The kernel implements this. The engine's storage driver names which filesystem does the work.*

- The union filesystem serves a read from the topmost layer that holds the file. A write goes to the writable directory.

A layer is one directory in this stack.

- The kernel describes an overlay filesystem as presenting the result of overlaying one filesystem on another [S74].
- Docker's driver names the three directories `lowerdir`, `upperdir` and `merged` [S6].
- The storage driver is a name for a kernel filesystem, not for a vendor invention [inference].

52 52. Layering is copy-on-write, one instruction at a time

On screen:

Part III · Question 2

The first write to an existing file copies it up from the image layer into the writable layer.

1

Share

Every container from one image reads the same layer directories.

2

Protect

The kernel mounts those directories read-only.

3

Copy on write

The first write to a file copies that file into the writable layer, and the write lands on the copy.

- The copy happens on the first write to each file, not on the first write overall.
- `fork(2)` runs this same protocol on memory pages instead of files.

- The first write to an existing file triggers a `copy_up` from the image layer [S6].
- The trap fires on the first write to each unit, not the first write overall [V19].
- `fork(2)` runs the same protocol at page granularity [V19].

- One build instruction produces one layer in the stack [S23].

53 53. The unit of sharing is a whole file

On screen:

Part III · Question 2

A union filesystem works at file level, not block level. The copy is never partial.

2 GB

the overlay driver copies when a process appends one byte to a 2 GB file

128

read-only layers the default overlay driver supports at once

1

copy of a file held in memory, shared by every container reading it

- *A database writing small records into an image layer therefore copies each file whole before its first change.*
- *It is also why fifty containers from one image cost one copy of that image's files in memory.*

- All `copy_up` operations copy the entire file, however small the change [S6].
- The overlay2 driver supports up to 128 lower layers [S6].
- "Multiple containers accessing the same file share a single page cache entry for that file" [S6].
- Modifying a large file in an image layer copies all of it first [V20].

54 54. The mount namespace, and what a volume really is

On screen:

Part III · Question 3

A mount namespace gives the process its own table of mount points.

Docker

[code]

Podman

[code]

1 · The kernel part

The mount namespace. Mount points added inside it are invisible outside.

2 · The engine part

Creating the directory, remembering its name, and asking for the mount.

3 · The consequence

A volume is a host directory placed into that table, so it survives the container.

- The mount namespace isolates mount points [S1].
- The OCI runtime bind-mounts volumes while setting the container up [V29].
- The engine says a volume is "completely managed by Docker" [S76].
- That wording describes the engine's bookkeeping and names no kernel object [inference].
- Volumes retain data after the containers using them are removed [S64].

55 55. Control groups: what it can consume

On screen:

Part III · Question 4

A control group organises processes into a hierarchy whose use of a resource can be limited and measured.

Docker

[code]

Podman

[code]

- *Version 2 collects every resource type in one tree, and only the tree's outermost groups may hold processes.*
- *The resource types that matter here are memory, processor time, process count and input or output.*

- A control group constrains what a process may consume. It changes nothing a process reads.

- Control groups organise processes into hierarchical groups whose resource use is limited and monitored [S2].
- Under version 2 "all mounted controllers reside in a single unified hierarchy" [S2].
- Version 2 requires that processes reside only in leaf nodes [S2].
- Resource limits on a container without root require version 2 [S61].

56 56. Namespaces hide, control groups constrain

On screen:

Part III · Question 4

Two mechanisms, two different verbs. The memory result needs both words to explain it.

Namespace

Changes what is visible

Process identifiers, mounts, hostname, network.

Nothing about consumption.

Control group

Changes what is allowed

Memory, processor time, process count.

Nothing about visibility.

The limit held because a control group enforced it. The report showed the host because no namespace covers that file.

- A namespace makes processes inside see their own instance of one resource [S1].
- A control group limits and measures resource usage [S2].
- The two are configured separately and do different work [V1].

57 57. Nobody gave `/proc/meminfo` a namespace

On screen:

Part III · Question 4

No kernel change fixes the memory report. A userspace workaround covers those paths instead.

- *`/proc/meminfo` and its neighbours always display numbers from the host system.*
- *Docker's own documentation states that `free` inside a container reports the host's swap.*
- *A separate project, LXCFS, mounts a small userspace filesystem over those paths so the numbers match the limit.*
- *It is a workaround in userspace, and no kernel change replaces it.*

- `/proc/meminfo`, `/proc/vmstat` and friends "are not containerized" and always show the host [S41].
- Docker states that `free` inside a container reports the host's available swap [S68].
- LXCFS is "a small FUSE filesystem written with the intention of making Linux containers feel more like a virtual machine" [S69].
- A container whose memory numbers match its limit is still one process on the host kernel [inference].

58 58. Capabilities: how much of root is left

On screen:

Part III · Question 5

Linux divides the privilege traditionally held by the superuser into distinct units, each switchable on its own. One such unit is a capability.

`CAP_SYS_ADMIN`

Mounting filesystems, setting the hostname, and creating namespaces. Close to root on its own.

`CAP_SYS_PTRACE`

Tracing arbitrary processes and reading their memory. Not in either engine's default set.

`CAP_NET_RAW`

Raw and packet sockets. In Docker's default set and not in Podman's.

- *The engine drops most capabilities before it starts the container, and you can print the set.*

- Linux divides superuser privilege into independently switchable capabilities [S3].
- `CAP_SYS_ADMIN` permits mounting filesystems, setting the hostname and creating namespaces [S3].
- `CAP_SYS_PTRACE` permits tracing arbitrary processes and reading their memory [S3].
- `CAP_NET_RAW` permits raw and packet sockets [S3].
- Docker's default set adds `CAP_NET_RAW` [S43], and Podman's eleven defaults do not include it [S42].
- Neither default set holds `CAP_SYS_PTRACE`: Podman's eleven [S42], or Docker's fourteen [S43].
- The manual page warns against assigning new features to `CAP_SYS_ADMIN`, whose scope is already excessive [S3].

59 59. Read the default set from the engine's own source

On screen:

Part III · Question 5

The default is not folklore. Both engines declare it in one file of their source code.

Podman

Eleven capabilities, from `CHOWN` to `SYS_CHROOT`, in `pkg/config/default.go`.

Docker

The same eleven plus `MKNOD`, `NET_RAW` and `AUDIT_WRITE`, in `daemon/pkg/oci/caps/defaults.go`.

Both

Neither list contains `SYS_PTRACE`.

Docker

[code]

Podman

[code]

- Podman's default set lists eleven capabilities and no `CAP_SYS_PTRACE` [S42].
- Docker's default set adds `CAP_MKNOD`, `CAP_NET_RAW` and `CAP_AUDIT_WRITE` to those eleven [S43].
- The capability set of a running process can be read from `/proc` [S3].

60 60. Seccomp: which calls it may issue

On screen:

Part III · Question 5

A seccomp filter is a small program the kernel runs before each system call. It returns allow, an error, a kill, or a notification to a tracer.

- *Docker's default profile refuses about 44 of more than 300 system calls.*
- *`mount` is refused, with the stated reason that the capability already gates it.*
- *Installing a filter needs `CAP_SYS_ADMIN`, or the `no_new_privs` setting, which stops the process gaining privilege later.*
- *The capability set and the filter are coupled in practice. Granting a capability also allowlists the calls that go with it.*

- The default profile "disables around 44 system calls out of 300+" [S5].
- `mount` carries the reason "Deny mounting, already gated by `CAP_SYS_ADMIN`" [S5].
- Installing a filter requires `CAP_SYS_ADMIN` or the `no_new_privs` bit [S4].
- In containerd, a process that has a capability is also given the system calls that go with it [S17].

61 61. The refusal named a mechanism

On screen:

*Part III · Question 5**A tracer attaches in two cases only. The Yama kernel module applies both.**1**Capability**The tracer holds `CAP_SYS_PTRACE` . The option grants it.**2**Relationship**Or the target is a descendant of the tracer.**3**Yama**Yama applies the limit that `kernel.yama.pttrace_scope` sets.**4**The filter**A seccomp filter refuses separately, on the call itself.**Read the refusal. `--privileged` switches every restriction off.*

- Yama is a Linux Security Module, and `kernel.yama.pttrace_scope` sets how far a tracer may reach [S84].
- Scope 1 is the default. There a tracer attaches only to its own descendants, unless it holds `CAP_SYS_PTRACE` or the target declared it a tracer [S84] [S67].
- At scope 2 only a holder of `CAP_SYS_PTRACE` may attach at all [S84].
- Under Yama's default setting, attaching requires `CAP_SYS_PTRACE` or a predefined relationship, by default a descendant of the caller [T5] [S67].
- The manual page names Yama as one enabled Linux Security Module, beside SELinux and Smack, and the tracing checks depend on it [S67].
- Adding the capability also allowlists `ptrace` , `process_vm_readv` , `process_vm_writev` and `kcmp` [S17].
- Docker's profile blocked `ptrace` only on kernels before 4.8 [S5].
- `--privileged` and a mounted engine socket are the absence of a boundary [V8].

62 62. One file holds all five requests

On screen:

Part III · The five together

One configuration file is written before a container starts. Every mechanism in this part has a field in it.

[code]

- *root* names the tree the union filesystem assembled. *process.capabilities* says how much of root is left.
- *namespaces* says what the process can see. *cgroupsPath* and *resources* say what it can consume. *seccomp* says which calls it may issue.
- *This is an excerpt. Each mechanism from this part has its own field, and one file describes one container.*

- The configuration's `root` object names the container's root filesystem path [S81].
- `process.capabilities` carries the capability sets of the container process [S81].
- The Linux section of the file defines `namespaces`, `cgroupsPath`, `resources` and `seccomp` [S7].
- The `namespaces` array names each namespace, and an optional `path` joins an existing one [S7].
- `cgroupsPath` names the control group [S7].
- `resources` carries the resource limits [S7].
- `seccomp` carries the system-call filter [S7].
- The OCI runtime reads this file and creates the namespaces [V29].
- The OCI runtime then attaches the control group and applies the SELinux labels [V29].
- The OCI runtime loads the seccomp filter and bind-mounts the volumes [V29].

63 63. A container is a list of requests

On screen:

Part III · The five together

There is no struct container in Linux, no call that creates one, and no subsystem that owns the word.

1 · Namespaces

What the process can see.

2 · Control group

What it can consume.

3 · Capabilities

How much of root it keeps.

4 · Seccomp filter

Which system calls it may issue.

5 · Union filesystem

What it sees as its root directory.

Remove any one and something still runs. You just have less of a container.

- A container is an ordinary process. Unrelated kernel interfaces restrict what it sees and what it consumes [T1] [V1].
- "Remove any one of those five and you still have something that runs" [V2].
- The question "is this a container?" has no answer, and "which of the five did this command request?" does [V2].
- The kernel maintainers declined the word, and renamed process containers to control groups [V6].

64 64. Two architectures

On screen:

Part IV

The same five requests. Two different answers to one question: which process owns the container?

Part IV compares process architectures, not kernel usage.

65 65. Engine, OCI runtime, and the work

On screen:

Part IV · The tiers

Two programs, two jobs. The engine writes the request. The OCI runtime issues it.

The engine

Resolves the image name.

Unpacks the layers.

Writes the configuration file.

The OCI runtime

Reads that file.

Creates the namespaces.

Attaches the control group.

Sets the capability set and installs the seccomp filter.

Mounts the volumes.

Starts the command.

- `runc` , `crun` and `youdi` are OCI runtimes. They implement one standard, published by the Open Container Initiative.
- Changing the engine does not change this tier. Both engines can use either OCI runtime.

- The OCI runtime creates the namespaces and attaches the control group [V29].
- The OCI runtime applies the SELinux labels and loads the seccomp filter [V29].
- The OCI runtime bind-mounts the volumes and forks the container process [V29].
- The three OCI runtimes are interchangeable implementations of one specification [V16].
- The Open Container Initiative publishes that runtime specification [S7] [S9].
- Podman's default OCI runtime is `crun` , selectable with the `--runtime` option [S13].

66 66. Docker: a client, a daemon, and two tiers

On screen:

Part IV · Docker

A daemon is a program that keeps running after the command that started it exits, and owns resources on the user's behalf.

[code]

- *The client and the daemon talk over an HTTP interface on a local socket file.*
- *The daemon builds, runs and distributes containers, and owns the images and volumes.*
- *A shim is a small process that containerd leaves in place to own one container.*
- *containerd was split out of the daemon to separate low-level container management.*

- "Docker uses a client-server architecture", talking over a REST API on UNIX sockets [S10].
- The daemon manages images, containers, networks and volumes [S10].
- Low-level container management was split into containerd in Docker Engine 1.11 [S22].
- Each shim instance "communicates with containerd as a daemon, while parenting containers via invoking independent runtimes" [S70].

67 67. What the host process table actually shows

On screen:

Part IV · Docker

The container is not a child of the daemon on a current installation. A small shim owns it.

Docker

[code]

Podman

[code]

Why

The daemon does not launch containers itself. A shim instance parents the container and invokes the OCI runtime.

What survives

Under Docker the container outlives your terminal, because nothing in your session owns it.

Read the process tree before trusting any drawing of it.

- "containerd, the daemon, does not directly launch containers" [S70].
- Each shim instance "communicates with containerd as a daemon, while parenting containers via invoking independent runtimes" [S70].
- The container outlives the terminal that started it [inference].

68 68. What the daemon actually controls

On screen:

Part IV · Docker

The daemon does not hold the container as a parent. It holds it by policy, and the policy has a default.

Claim that is wrong

The daemon is the parent

The process tree shows a shim, not the daemon.

Claim that holds

The daemon controls the lifecycle

By default, when the daemon stops it shuts running containers down.

Live restore changes that, and it is not the default.

Docker supports live restore only when you install a patch release.

- The daemon can be configured "so that containers remain running if the daemon becomes unavailable" [S12].
- Live restore is "only supported when installing patch releases" [S12].

- Each shim instance parents containers by invoking independent runtimes [S70], so parentage and lifecycle policy are separate claims [inference].

69 69. Podman: fork and exec, with a monitor left behind

On screen:

Part IV · Podman

Podman has no daemon. Each command is an ordinary program that sets the container up and starts the OCI runtime.

[code]

- The `podman` process itself exits once the container is running.
- A small program named `conmon` stays behind. It started the OCI runtime, and it sits between your session and the container.
- The container is therefore a child of your session or of a `systemd` unit.

- Podman is "a fully featured container engine that is a simple daemonless tool" [S13].
- Most Podman commands run as a regular user without extra privileges [S13].
- `conmon` execs `crun` or `runc`, which sets the container up and forks the container process [V29].
- `conmon` therefore remains between the user's session and the container process [inference].

70 70. What a start without root actually does

On screen:

Part IV · Podman

Removing the daemon moves work into userspace. Podman does it in a fixed order.

1 · Pause process

Keeps one user namespace alive for all your containers.

2 · Identifier map

Reads your account's reserved user numbers and maps them.

3 · Storage

Makes a mount namespace and assembles the layers.

4 · Network

Starts a userspace network stack. The kernel refuses a full one.

5 · OCI runtime

Execs `common` , then the OCI runtime.

- Podman joins the pause process namespace "because a multithreaded program cannot change its user namespace" [S15].
- Without that process Podman reads `/etc/subuid` and `/etc/subgid` , builds the mapping, and forks the pause process [V32].
- Podman then creates a mount namespace and assembles the image layers [V33].
- Full separate networking is not allowed for non-root users, so a userspace stack emulates it [S15].
- The OCI runtime then creates the namespaces and attaches the control group [V29].
- The OCI runtime applies the SELinux labels and loads the seccomp filter [V29].
- The OCI runtime bind-mounts the volumes last [V29].

71 71. Predict: whose trace shows the namespace call?

On screen:

Part IV · Which program asks the kernel

Write your answer down. Run these in the Linux shell from the setup, as your own user.

Docker

[code]

Podman

[code]

Question 1

In which of the two traces do you see `unshare` ?

Question 2

Which program makes the call: the one you typed, or a descendant of it?

- *-Y prints the program name beside each process number. The `-e trace= list` keeps the trace short.*
- *Install the tracer inside the Linux shell first: `sudo apt install strace`, or `sudo dnf install strace`.*

- `unshare(2)` moves the caller into a new namespace, and `setns(2)` joins one by descriptor [S1].
- A written prediction before the observation is the method this course uses [S57].
- Podman execs `common`, and `common` execs the OCI runtime [S15].
- Docker's client talks to a daemon over a socket, so the work happens in another process tree [S10].

Collect the answers first.

72 72. Observe: only one of the two traces holds the call

On screen:

Part IV · Which program asks the kernel

Docker

[code]

Podman

[code]

- *Under Podman, one call creates five namespaces at once.*
- *The program that makes it is `runc:[1:CHILD]`, a descendant of the `podman` command you typed.*
- *Under Docker, the client only opens a socket to the daemon.*
- *The same call happens under the daemon, in a process tree your trace never entered.*

- The OCI runtime creates the namespaces for the container [V29].
- Podman execs `common`, and `common` execs the OCI runtime [S15], so the runtime is a descendant of the command you typed [inference].

- `unshare(2)` takes one or more `CLONE_NEW*` flags and moves the caller into new namespaces [S1].
- Docker's client and daemon talk over a REST API on a UNIX socket [S10].
- Nothing in the client's own trace therefore creates a namespace [S10] [inference].

73 73. Explain: the call is the same, the parent is not

On screen:

Part IV · Which program asks the kernel

Docker

[code]

Podman

[code]

- Attached to `containerd`, the trace shows the same `runc:[1:CHILD]` calling `unshare`.
- Yama allows a tracer only its own descendants. The daemon's tree is not one, even where it runs as your account, so the attach needs `sudo`.
- Flag lists differ between versions. Read your own trace rather than this one.

Two architectures, one system call. What differs is whose child makes it.

- Each shim instance parents containers by invoking independent runtimes under `containerd` [S70].
- The OCI runtime is what creates the namespaces, under either engine [V29].
- Under Yama's default setting a tracer is restricted to its own descendants, unless it holds `CAP_SYS_PTRACE` [S84] [S67].
- `containerd` is a long-lived service, so its tree is not a descendant of the user's shell and the attach needs privilege [S70] [S84] [inference].
- The flag set differs between engine and kernel versions, so a trace is read and not memorised [inference].

74 74. Two OCI runtimes, one specification

On screen:

Part IV · The OCI runtimes

runc is written in Go. *crun* is written in C. They implement the same standard and are interchangeable.

3.34 s

100 sequential containers under *runc*, in the *crun* project's own benchmark

1.69 s

the same 100 under *crun*, same benchmark

512 KB

memory limit *crun* still ran under. *runc* failed even with 4 MB

- These are the project's own figures on its own benchmark. Read them as directional.
- *youki* is a third OCI runtime built to the same standard.

- *crun* is "a fast and low-memory footprint OCI Container Runtime fully written in C" [S16].
- The 1.69-second and 3.34-second figures, and the 512 KB result, are the project's own [S16].
- *runc*, *crun* and *youki* are interchangeable implementations of one specification [V16].

75 75. Rootless: two different concessions

On screen:

Part IV · Without root

Rootless means both the engine and the container run as an unprivileged user, on top of a user namespace.

Docker

Keeps the daemon, moves it

The daemon and the containers both run inside a user namespace.

The daemon itself runs unprivileged too.

It needs at least 65,536 reserved user numbers for your account.

Podman

Has no daemon to move

Rootless is the ordinary way to run it.

Podman states it holds no more privilege than the account that started it.

Part VI shows a kernel flaw that no identifier mapping stops.

- Rootless mode runs "the Docker daemon and containers inside a user namespace" [S11].
- In rootless mode "both the daemon and the container are running without root privileges" [S11].
- It requires at least 65,536 subordinate identifiers for the user [S11].
- Rootless Podman builds its identifier mapping from `/etc/subuid` and `/etc/subgid` [V32].
- Podman states that rootless containers "will never have more privileges than the user that launched them" [S14].
- The sentence is a design statement, not a measurement [inference].
- Dirty Pipe let an unprivileged process write to any file it could read [S30].
- The published proof of concept escaped a container that was granted no extra privilege [S30].
- An identifier map does not change a kernel code path, so it does not stop that flaw [inference].

76 76. Host policy overrides every engine option

On screen:

Part IV · Without root

Running without root rests on the user namespace. A Linux distribution can switch that off.

[code]

- *Ubuntu restricts unprivileged user namespaces by default since release 23.10, and this course's Windows path installs Ubuntu.*
- *A program without root may create a user namespace in two cases only:*

- A Linux security module named AppArmor confines the program, and its AppArmor profile carries the `usersns` rule.
- The program holds `CAP_SYS_ADMIN`.
- The switch is a kernel parameter on the host. No engine option reaches it.

- Ubuntu announced the restriction on unprivileged user namespaces with release 23.10 [S77].
- Ubuntu 24.04 "restricts the use of unprivileged user namespaces" by default [S48].
- Unprivileged processes may create them only if they "are confined and have the 'usersns,' rule in their AppArmor profile (or if they have `CAP_SYS_ADMIN`)" [S77].
- Rootless Podman rests on exactly that system call, so host policy gates it [inference].

77 77. The network path without root is a userspace stack

On screen:

Part IV · Without root

An unprivileged user cannot create the full network path. A program in userspace copies packets instead.

17.9 Gbps

one iperf3 stream, container running as root

570 Mbps

the same stream, container without root, in one study

- The study's authors attributed the gap to per-packet handling in `slirp4netns`.
- Podman 6 removed that stack. The shipped default is now `pasta`.
- Publishing a port without root is a userspace listener, not a firewall rule.

- One study measured 570 Mbps against 17.9 Gbps and attributed the gap to `slirp4netns` [V14].
- Podman 6.0.0 removed `slirp4netns` and directs users to Pasta [S65].
- The shipped configuration now sets the rootless network command to `pasta` [S78].
- Both engines use a userspace stack for this path [V28].

78 78. One image, either engine

On screen:

Part IV · One format

The image format and the OCI runtime specification are written standards, not one vendor's implementation detail.

Docker

[code]

Podman

[code]

- *The same registry, the same manifest, the same layer archives for both.*
- *The manifest names each layer archive by the hash of its content, and any engine can unpack them.*
- *A registry serves the same image to both engines over the same protocol.*
- *This is why a second engine can be a replacement rather than a copy.*

- The image manifest names "the content-addressable identity of one or more filesystem layer changeset archives" [S8].
- The runtime specification defines the configuration every OCI runtime reads [S7].
- Podman keeps the OCI image format and a deliberately compatible command set [S13].

79 79. The disagreement is about ownership

On screen:

Part IV · The summary

Both engines make the same five requests to the same kernel. They disagree about which process holds the container.

	DOCKER	PODMAN
<i>Path</i>	<i>client to daemon to containerd to shim to runc</i>	<i>one command, common , then crun</i>
<i>Owner</i>	<i>a long-lived privileged program</i>	<i>your own session, or a systemd unit</i>
<i>Default OCI runtime</i>	<i>runc</i>	<i>crun</i>
<i>Without root</i>	<i>supported, daemon moved into a user namespace</i>	<i>the ordinary mode</i>
<i>Default capabilities</i>	<i>eleven, plus MKNOD , NET_RAW and AUDIT_WRITE</i>	<i>eleven</i>
<i>Builds</i>	<i>one interface always listening, an integrated build system</i>	<i>least privilege by default, no idle program</i>
<i>Costs</i>	<i>a privileged program on the path of every container</i>	<i>userspace shims where the kernel refuses</i>

- The disagreement is about who owns the container process [T3] [V15].
- Docker's path runs through a daemon and containerd [S10] [S22].
- Podman execs `common` , which execs the OCI runtime and stays behind as the monitor [S15].
- Neither engine asks the kernel for a different mechanism [V1].
- Their default capability sets do differ: Docker's fourteen against Podman's eleven [S42] [S43].
- Podman's default OCI runtime is `crun` [S13]. Docker reaches `runc` through the shim named `containerd-shim-runc-v2` [S70] [inference].
- Docker runs without root by moving the daemon and the containers into a user namespace [S11].
- Most Podman commands run as a regular user without extra privilege, so this is its ordinary mode [S13].
- The Docker daemon listens for API requests on a socket, and it also builds and distributes images [S10].
- Rootless mode is what runs Docker's daemon without root privileges, and `usersns-remap` mode keeps it privileged [S11].
- The daemon therefore sits on the path of every Docker container, and it holds root privileges unless rootless mode runs it [S10] [S11] [inference].

- Podman runs no daemon, so no long-lived program waits between commands [S13].
- Podman's default set holds three fewer capabilities than Docker's, so its container starts with less privilege [S42] [S43].
- Without root the kernel refuses full networking to Podman, so a userspace program carries the packets instead [S15].

80 80. What follows from that

On screen:

Part V

Two consequences of Part IV: the commands are portable, and macOS and Windows needed a Linux virtual machine.

Part V draws consequences. No new mechanism.

81 81. Same verbs, different engine

On screen:

Part V · Consequence 1

Part II showed the commands for one engine. Almost all of them run on the other unchanged.

TASK	DOCKER	PODMAN
Run	<code>docker run</code>	<code>podman run</code>
List	<code>docker ps</code>	<code>podman ps</code>
Build	<code>docker build</code>	<code>podman build</code>
Compose	<i>built in</i>	<i>needs an external provider installed</i>
Trace	<code>--cap-add SYS_PTRACE</code>	<code>--cap-add SYS_PTRACE</code>

- *The verbs match because the image format and the OCI runtime specification are standards.*
- *Compose is the exception. Podman ships a wrapper and no implementation.*

- Podman keeps the OCI image format and a deliberately compatible command set [S13].
- "podman compose is a thin wrapper around an external compose provider" [S66].
- Podman states that a container without root never holds more privilege than the user who launched it [S14].

82 82. Why macOS and Windows need a Linux virtual machine

On screen:

Part V · Consequence 2

A virtual machine runs its own kernel, so the guest kernel is not the host kernel. That is what macOS and Windows are missing.

1 · The mechanisms

All five are Linux kernel features.

2 · The gap

The macOS and Windows kernels lack them.

3 · The answer

Supply a Linux kernel in a virtual machine.

4 · Your own case

Lima and WSL 2 are instances of this, not workarounds.

Microsoft says so plainly: WSL 2 is a Linux kernel in a small virtual machine.

- Every Docker on a Mac is Docker inside a Linux virtual machine [T4] [V9].
- WSL 2 uses "a Linux kernel inside of a lightweight utility virtual machine (VM)" [S46].
- WSL 2 also "runs Linux distributions as isolated containers inside the managed VM" [S46].
- The mechanisms named are Linux kernel interfaces [S1] [S2] [S3].

83 83. Where "a Linux virtual machine" stops being the answer

On screen:

Part V · Consequence 2

The claim holds for Linux containers. Windows has containers of its own, and they need a Windows kernel.

Linux container on Windows

Needs a Linux kernel

Supplied by WSL 2, inside a virtual machine.

Everything in this topic applies.

Windows container on Windows

Needs the Windows kernel

It shares the Windows kernel directly. Microsoft describes that as roughly how Linux containers run.

Adding a Linux virtual machine does not help here.

- Process-isolated Windows containers "share the same kernel with the host as well as each other" [S35].
- Microsoft calls that "approximately the same as how Linux containers run" [S35].
- The takeaway is scoped to Linux containers for this reason [inference].

84 84. One shared machine, or one for each container

On screen:

Part V · Consequence 2

How many virtual machines the engine builds is a design choice, and the answers differ.

Lima and WSL 2

Lima, WSL 2

One virtual machine. Every container is a process inside it.

One kernel, many containers

Apple

its Containerization project

Each Linux container runs inside its own small virtual machine.

Stronger separation, more machines

- *On the second shape two containers share no kernel, so the shared-kernel claim describes the usual shape and not a law.*

- Apple states that "Containerization executes each Linux container inside of its own lightweight virtual machine" [S37].
- The Lima and WSL 2 path runs one virtual machine for many containers [V9].
- Part VI states the shared-kernel argument for the usual shape and names this exception [inference].

85 85. Where it breaks

On screen:

Part VI

The boundary, the escapes through it, and the leaks you can measure on your own laptop.

Part VI is the diagnostic part. Evidence first, judgement after.

86 86. The boundary is the system-call interface

On screen:

Part VI · The boundary

A virtual machine gives a guest its own kernel behind a hardware boundary. A container shares the host kernel, so its boundary is that kernel's system-call interface.

1

kernel shared by every container on the host

300+

system calls the boundary is drawn on

~44

of them refused by Docker's default profile

- *Linux publishes a stable, versioned, documented system-call interface.*
- *Stability makes it dependable. Width makes it large to defend.*
- *A kernel bug is therefore a possible container escape.*

- Namespaces separate a container more weakly than a hardware-backed virtual machine separates a guest [T2] [V7] [S73].
- The attack surface takes in the kernel's whole system-call interface [V7].
- The kernel's stable, versioned, documented user-space interface is the numbered system-call interface [V21].
- The default profile refuses around 44 of more than 300 calls [S5].
- Kubernetes states that containers "offer a weaker isolation boundary than virtual machines that utilize hardware-based virtualization" [S73].

87 87. A container escape, in four steps

On screen:

Part VI · The escapes

CVE-2019-5736 overwrote the host OCI runtime binary from inside a container.

1

*Bait**A malicious image points its entry at `/proc/self/exe` .**2**Re-entry**The OCI runtime re-executes itself inside it.**3**Handle**A library the attacker loaded opens the host's OCI runtime binary for writing.**4**Overwrite**The next start runs attacker code as root.*

- *The attack needed only the default options.*
- *It reached Docker, Podman, CRI-O and containerd alike.*

- The entry point is a symlink to `/proc/self/exe` , so the OCI runtime re-executes itself inside the container [S25].
- An `LD_PRELOAD` ed library constructor "can obtain a writable descriptor to the host binary" [V34].
- The flaw "let a malicious image overwrite the host `runc` binary" [V34].
- The engine executes that same host `runc` binary again at every later container start [V34] [inference].
- Rootless mode is what runs Docker's daemon without root privileges, and `usersns-remap` mode keeps it privileged [S11].
- Docker's daemon binds a UNIX socket owned by root, so a default install runs it with root privileges [S85] [inference].
- The OCI runtime that this privileged path executes therefore runs as root too [inference].
- The next start therefore executes the attacker's code as root [inference].
- The escape needed only the default options and a standard configuration [V37].
- The documented response is to treat the kernel as belonging to a single tenant [S25].
- The flaw reached Docker, Podman, Kubernetes, CRI-O and containerd alike [S24].

88 88. One class, disclosed four times

On screen:

Part VI · The escapes

One class of bug returned four times in `runc`. The container controls a path, and the OCI runtime follows it.

Nov 2025

Three flaws of one class. Each turned on a path the container controlled.

One mechanism

A link replaces `/dev/null`, so the OCI runtime mounts a host path read-write.

Another

The kernel runs a helper on the host, and that helper ignores namespaces.

2026

CVE-2026-41579. Again a `/dev` link, again limited host writes.

- An untrusted image or build file is the most likely way in. The `runc` 1.1.x branch stayed unpatched.*

- November 2025 disclosed three CVEs of one class: CVE-2025-31133, CVE-2025-52565 and CVE-2025-52881 [V36].
- Those three abused masked paths and `/dev/console` mount races [V36].
- Each of the four turned on a path the container itself controlled, which is what makes them one class [inference].
- `runc` is the OCI runtime Docker, Podman, Kubernetes, CRI-O and containerd all use [S24].
- The fixes shipped in `runc` 1.2.8, 1.3.3 and 1.4.0-rc.3 [S26].
- The `runc` 1.1.x branch and everything earlier were left unpatched [S26].
- CVE-2026-41579 is the fourth disclosure of the same class [S28] [V36] [inference].
- In CVE-2025-31133, replacing `/dev/null` with a symlink makes `runc` bind-mount an attacker-chosen host path read-write [S26].
- "As kernel upcalls are not namespaced", the configured binary runs with full host privileges [S26].
- CVE-2026-41579 "allowed a malicious image with a `/dev` symlink to have limited write access to the host filesystem" [S28].

- Untrusted images and build files are the "most likely delivery method" [S27]. Youki and crun "are understood to have similar flaws" [S27].

89 89. When the kernel is the bug

On screen:

Part VI · The escapes

Dirty Pipe was a kernel flaw. No fix in an OCI runtime could have prevented it.

- *CVE-2022-0847 let an unprivileged process write to any file it could read.*
- *A public proof of concept escaped a container that was granted no extra privilege. It overwrote the cached copy of the OCI runtime binary.*
- *Patching the engine would not have helped. Changing the engine would not have helped.*
- *This is what "the boundary is the kernel" means in practice.*

Every process in every container on that host was behind the same one bug.

- Dirty Pipe allowed an unprivileged process to write to any file it could read [S30].
- The published proof of concept escaped a container that was granted no extra privilege, through the page-cache copy of `runc` [S30].
- No fix in the OCI runtime could have prevented a kernel bug [inference].

90 90. One systematic study, and what it counted

On screen:

Part VI · The evidence

One published survey counted the flaws across eleven implementations instead of estimating them.

59

published flaws across the 11 "container runtimes" one survey examined

46%

of the 28 that had a working proof of concept enabled an escape

7 of 9

tested escapes prevented by user namespaces

- *The survey's authors put most cases in one category: a host component leaked into the container.*
- *A user namespace prevented seven of the nine exploits the survey tested. Two still worked.*
- *The two that survived involved the host OCI runtime still running as root against objects the container controls.*

- The study covers 59 flaws across the 11 "container runtimes" it examined, 28 of them with public proof-of-concept exploits [S29].
- 46 percent of those enabled container escape, and the dominant cause was a leaked host component [S29].
- User namespaces prevented 7 of 9 tested escape exploits, so two of the nine still worked [S29].
- The two exploits that survived did so because the host OCI runtime still executes as root against container-controlled objects [V35].
- An LXC maintainer who worked on the runc fix states that privileged containers "are simply never going to be truly secure" [S24].

91 91. The other boundary is narrower, not absolute

On screen:

Part VI · The comparison

A virtual machine's interface is smaller than a system-call interface. Smaller is not zero.

- *CVE-2025-22225 is a published flaw in VMware ESXi. An attacker with enough privilege could reach the host through the process that runs the virtual machine.*
- *The comparison is about the width of the interface, not about a guarantee on either side.*

Container

The whole system-call interface of a shared kernel.

Virtual machine

A much smaller device and instruction interface. An attacker crossed it too.

- For CVE-2025-22225 "an attacker with requisite privileges could exploit this vulnerability through the VMX process to escape the sandbox" [S36].
- Kubernetes states the container boundary is weaker than hardware-based virtualization [S73].
- The claim is comparative width, not a guarantee on either side [inference].

92 92. Containers went back inside virtual machines

On screen:

Part VI · The comparison

AWS replaced Linux containers with microVMs to isolate the functions its on-demand service runs.

- *AWS first isolated its on-demand function service with Linux containers, and reported itself unsatisfied.*
- *Their stated reason: reliance on a single kernel forces a trade-off between security and compatibility.*
- *They built Firecracker instead of choosing between the two.*
- *Kubernetes says sandboxing is often recommended for untrusted code, where workloads are assumed to be malicious.*

Packaging is where containers replaced the alternative. Untrusted tenants are where they did not.

- AWS replaced Linux containers with Firecracker microVMs for the functions Lambda runs [S38].
- "When we first built AWS Lambda, we chose to use Linux containers to isolate functions... We were unsatisfied with this approach" [S38].
- Reliance on a single kernel means "a fundamental tradeoff between security and code compatibility" [S38].
- AWS states "We built Firecracker because we didn't want to choose" [S38].
- Kubernetes states that sandboxing "is often recommended when you are running untrusted code, where workloads are assumed to be malicious" [S73].

- Part I named multi-tenancy as the first need, and this is where it landed [inference].

93 93. Which containers are not virtual machines

On screen:

Part VI · The scope

The shared-kernel claim covers the usual shape. Two documented cases fall outside it.

Kata Containers

project page

An OCI runtime that builds a small virtual machine for each container.

An engine runs it instead of `runc`

The standard

the OCI runtime specification

The runtime specification carries a virtual-machine section of its own.

Not a deviation

- *So the claim is scoped. A namespace-based Linux container shares the host kernel, and an OCI runtime that builds a virtual machine still conforms to the same standard.*

- Kata Containers is "An open source container runtime, building lightweight virtual machines" [S33].
- An engine's `--runtime` option takes any OCI-compatible binary, so a conforming runtime replaces the default one [S13].
- The runtime specification "describes the schema for the virtual-machine-specific section of the container configuration" [S34].
- The claim is therefore scoped to namespace-based Linux containers [inference].

94 94. The leak you can measure: files across the boundary

On screen:

Part VI · The leaks

On macOS a bind mount crosses out of the virtual machine on every access. A volume does not.

3.5x

slower than Linux with VirtioFS, many small files through a bind mount, in one measurement

10x

slower on the same files with gRPC-FUSE

~1x

in line with Linux, inside a named volume

- *One author measured these ratios on one codebase. They show direction only.*
- *Many small files dominate the cost, not total bytes.*
- *On Windows, keep working files in the Linux filesystem.*
- *Two machines on one engine version differ if one uses the slower file-sharing driver.*

- Every container on macOS runs inside a Linux virtual machine, so a bind mount crosses that boundary [V9] [inference].
- One measurement puts macOS at roughly 3.5 times slower with VirtioFS and 10 times with gRPC-FUSE [S44].
- Moving the directory into a named volume brought timings back in line with Linux [S44].
- Microsoft advises storing files on the Linux drive for speed [S72].
- Storage carrying a `.has-mount-program` flag file keeps using FUSE whatever the kernel [V13].
- The program leaves the virtual machine on every access to a bind-mounted path, and the measurement tracks that crossing [V9] [S44] [inference].

95 95. The leak you can trip over: the wrong architecture

On screen:

Part VI · The leaks

An image built for one processor architecture on a machine with another runs under translation, or not at all.

Clear failure

`exec format error` . The image was built for another architecture and nothing translates it here.

Unclear failure

`Illegal instruction` . The translator does not implement an instruction the program uses.

Slow success

It runs. Each system call is caught, converted and forwarded, so system-call-heavy work suffers most.

Fix

Name your own architecture with `--platform linux/arm64` or `--platform linux/amd64` . Prefer images built for that architecture.

- `exec format error` is the canonical symptom of an image built for the wrong architecture [S59].
- Rosetta does not implement AVX, so such images fail with `Illegal instruction` [S45].
- Under translation each system call is caught, converted and forwarded to the host kernel [S59].
- `--platform=OS/ARCH` selects the image for a given platform [S61].

96 96. What is inside an image nobody opened

On screen:

Part VI · The leaks

An image freezes a package set on the day it was built, and nothing refreshes it afterwards.

- *One scanning study of images built for neuroscience data analysis found they contain hundreds of known vulnerabilities.*
- *Rebuilding the same images against current packages removed about two thirds of those vulnerabilities.*

- *The same paper reports, from earlier work rather than its own, that over 30 per-cent of official images on one registry carry high-priority vulnerabilities.*
- *The images pulled in Part II are somebody else's filesystem, running under your en-gine.*
- *Rebuild on a schedule, because nothing else refreshes the package set.*

- The study measured images for neuroscience data analysis and found they "con-tain hundreds of vulnerabilities" [S31].
- Software updates removed "roughly two-thirds of these" [S31]. The 30-percent fig-ure for official images is quoted by that paper from earlier work [S31].
- The 2025 advisories name untrusted images as the most likely delivery method [S27].
- Image layers are read-only below the container's writable layer [S6], so only a new build applies those updates [inference].

97 97. Three things the evidence leaves open

On screen:

Part VI · Honest limits

•

A namespace can outlive its processes

namespaces(7)

Bind-mounting an entry in `/proc` keeps a namespace alive after every process in it exits.

•

The memory report has no kernel fix

Docker docs

Nothing namespaced those files. The workaround is a userspace filesystem.

•

Your own installation has its own defaults

containers.conf

Read your network backend and storage driver from `containers.conf`.

Read the real limit from `/sys/fs/cgroup/memory.max`, not `free`. Check `/proc` for a bind mount before trusting that a namespace is gone.

- Bind mounting a file in `/proc/pid/ns` "keeps the corresponding namespace... alive even if all processes currently in the namespace terminate" [S1].
- `/proc/meminfo` and its neighbours always display host numbers [S41].
- The control group holds the limit, and the files `free` reads do not [S2] [S41].
- The remaining default is read from `containers.conf` and the release notes for the installed version [V27].
- Where an inside number and an outside number disagree, count the boundary crossings between them [V23].

98 98. Where it came from

On screen:

Part VII · Optional

Four pictures. Optional.

Part VII is optional. Skip it without loss.

99 99. One line, 1982 to 2015

On screen:

Part VII · Optional

[code]

- `chroot` shows a process one subtree of the file tree and hides the rest.
- FreeBSD jails, in 2000, answered the same need with a compartment that reached past the file tree.
- Solaris Zones, in 2004, answered the same need on Solaris, and limited what each compartment consumed.
- The `chroot` date comes from a reading of the source-control logs, not from folklore.

- Popular timelines print 1979. The two accounts genuinely disagree, and the log date is better evidenced.
- Control groups are dated by the kernel release that carried them, not by a press date.

- `chroot(2)` was added to BSD by Bill Joy on 18 March 1982, per Kirk McKusick's reading of the SCCS logs [V30].
- `chroot` limits a process to a single subtree of the filesystem name space [S19].
- The compartment of `chroot` "does not extend to the process or networking spaces", and a jail does [S19] [inference].
- The jails paper states that a jail "effectively amounts to a virtual machine environment" [S19].
- The Solaris Zones paper states the need as isolation plus a limit on resource use, which is what a zone provides [S58] [inference].
- The author of FreeBSD jails states that `chroot` was never intended as a security mechanism [S20].
- Popular timelines date the call to 1979 and to Version 7 Unix, and the two accounts genuinely disagree [V31].
- The mount namespace arrived in kernel 2.4.19 in 2002 [S21].
- Control groups version 1 appeared in Linux 2.6.24 [S2].
- The 2013 date for the first Docker image and registry is the one every account cited here gives [inference].
- The Open Container Initiative was launched on 22 June 2015 [S9].

100 100. Three lanes that met

On screen:

Part VII · Optional

[code]

- The lanes are parallel answers to one pressure, not a line of descent. Accounts written at the time attribute the idea of namespacing to Plan 9, not to BSD.

• The kernel maintainers refused the word. Google's "process containers" became control groups.

- Jails, Zones and namespaces are three parallel responses rather than one inherited design [V5].
- Contemporaneous accounts attribute the idea of namespacing to Bell Labs' Plan 9 [V5].
- Google engineers began "process containers" in 2006, and the name changed to control groups to avoid overloading the word container [V6].
- Jails were presented as amounting to a virtual machine environment, which is where the reading began [S19].

101 101. What 2013 added, and what it did not

On screen:

Part VII · Optional

Isolation was available for a decade before the first Docker release. What was missing was a way to move an application between machines.

Not new in 2013

The isolation

Namespaces from 2002, control groups from 2008.

LXC had already combined them, and Docker began as a command-line front end to LXC.

New in 2013

The image and the transport

A layered image named by the hash of its content.

A protocol for moving it between machines.

Deployment becomes one command against a named hash.

- The problem containers were built to solve is distribution, not isolation [V3].
- The technically interesting part was the least technically novel part [V4].
- Docker began as a Python command-line front end to LXC and AUFS [S22].
- Low-level container management later moved out of Docker into containerd [S22].

102 102. Kubernetes dropped Docker, and the images kept working

On screen:

Part VII · Optional

Since 2015 the image format and the runtime contract are standards. One consequence arrived seven years later.

- *Kubernetes removed `dockershim`, its adapter for Docker, in release 1.24.*
- *Images built by Docker kept working everywhere, because they are standard OCI images.*
- *Docker was replaced underneath the format. The format did not move.*

Docker no longer controls the implementation, and the images it builds still run under every conforming engine.

- The v1.24 release of Kubernetes "actually removed the dockershim", the adapter for Docker [S39].
- "The image that Docker produces isn't really a Docker-specific image—it's an OCI image" [S40].
- A standard image therefore keeps working under every conforming engine [S40] [inference].
- The standard was launched in June 2015 by Docker, CoreOS and others [S9].

103 103. Takeaways

On screen:

1

No kernel object

Five mechanisms in one configuration file. Remove one and something still runs.

2

Not a small machine

The boundary is a shared kernel. The record has real escapes through it.

3

*Same parts, two shapes**They differ over who owns the container, not over what is asked of the kernel.*

4

*Linux, or a Linux inside**Lima and WSL 2 supply that kernel. Windows containers are the exception.*

5

*Configuration, not magic**Read the capability set before reaching for an option that removes it.*

- A container is a list of kernel requests, not a kernel object [T1] [V2].
- The boundary is the system-call interface of a shared kernel [T2] [V7].
- The two engines disagree about ownership of the container process [T3] [V15].
- A Linux container on macOS or Windows runs inside a Linux virtual machine [T4] [V9].
- What a process may do is a readable capability set and a readable filter [T5] [V10].

104 104. What follows

On screen:

*Four things to do differently once a container stops being a black box.**1 · Read the refusal**Find which capability or filter rule refused. Then grant that one.**2 · Choose where data lives**Writable layer, volume or bind mount. Decide before the first run.**3 · Check the host, not the flag**Control group version, namespace policy and architecture set the limits. No option overrides them.**4 · Keep the habit**Predict, observe, explain.*

Three of these read a value the machine already holds: the capability set, the storage destination, and the control group version.

- Never answer a permission error with `--privileged`, which is the absence of a boundary [V8].
- Treat root within a container as if it is root outside of the container [S32].
- The three storage destinations have three different lifetimes [S64].
- The capability set of a running process is readable from `/proc` [S3].
- Host policy gates rootless operation, and no engine option reaches it [S48] [S77].
- Podman 6.0.0 removed support for control groups version 1, so the host's version decides whether the engine runs at all [S65].
- Resource limits on a rootless container need control groups version 2 [S61].
- `exec format error` is the symptom of an image built for the host's other architecture [S59].
- `--platform=OS/ARCH` selects the image for a platform, and it cannot change the host's own [S61].
- The same correction applies to another everyday word: a thread is not a kernel concept [V26].
- The course toolchain is one reproducible environment built from these primitives [V24].

105 105. References

On screen:

- [S1] `namespaces(7)` — Linux manual page — <https://man7.org/linux/man-pages/man7/namespaces.7.html>
- [S2] `cgroups(7)` — Linux manual page — <https://man7.org/linux/man-pages/man7/cgroups.7.html>
- [S3] `capabilities(7)` — Linux manual page — <https://man7.org/linux/man-pages/man7/capabilities.7.html>
- [S4] `seccomp(2)` — Linux manual page — <https://man7.org/linux/man-pages/man2/seccomp.2.html>
- [S5] `Seccomp security profiles for Docker` — <https://docs.docker.com/engine/security/seccomp/>

- **[S6]** Use the OverlayFS storage driver — <https://docs.docker.com/engine/storage/drivers/overlayfs-driver/>
- **[S7]** OCI Runtime Specification — Linux Container Configuration — <https://github.com/opencontainers/runtime-spec/blob/main/config-linux.md>
- **[S8]** OCI Image Format Specification — <https://github.com/opencontainers/image-spec/blob/main/spec.md>
- **[S9]** About the Open Container Initiative — <https://opencontainers.org/about/overview/>
- **[S10]** Docker overview — <https://docs.docker.com/get-started/docker-overview/>
- **[S11]** Rootless mode — <https://docs.docker.com/engine/security/rootless/>
- **[S12]** Live restore — <https://docs.docker.com/engine/daemon/live-restore/>
- **[S13]** podman(1) — Podman manual page — <https://docs.podman.io/en/latest/markdown/podman.1.html>
- **[S14]** podman — project README — <https://github.com/podman-container-tools/podman>
- **[S15]** What happens behind the scenes of a rootless Podman container? — <https://www.redhat.com/en/blog/behind-scenes-podman>
- **[S16]** containers/crun — <https://github.com/containers/crun>
- **[S17]** Why strace doesn't work in Docker — <https://jvns.ca/blog/2020/04/29/why-strace-doesnt-work-in-docker/>
- **[S18]** What even is a container — <https://jvns.ca/blog/2016/10/10/what-even-is-a-container/>
- **[S19]** Jails: Confining the omnipotent root — <http://phk.freebsd.dk/pubs/sane2000-jail.pdf>
- **[S20]** Jails – High value but shitty Virtualization — <http://phk.freebsd.dk/sagas/jails/>
- **[S21]** Namespaces — <https://k1024.org/papers/namespaces.html>
- **[S22]** From dotCloud to Docker — <https://jpetazzo.github.io/2017/02/24/from-dotcloud-to-docker/>
- **[S23]** Docker build cache — <https://docs.docker.com/build/cache/>
- **[S24]** A container-confinement breakout — <https://lwn.net/Articles/781013/>
- **[S25]** Anatomy of CVE-2019-5736: A runc container escape — <https://aws.amazon.com/blogs/compute/anatomy-of-cve-2019-5736-a-runc-container-escape/>
- **[S26]** runc security advisory GHSA-9493-h29p-rfm2 — <https://github.com/opencontainers/runc/security/advisories/GHSA-9493-h29p-rfm2>

- **[S27]** *runc container breakout vulnerabilities: a technical overview* — <https://www.cncf.io/blog/2025/11/28/runc-container-breakout-vulnerabilities-a-technical-overview/>
- **[S28]** *runc releases* — <https://github.com/opencontainers/runc/releases>
- **[S29]** *Towards Improving Container Security by Preventing Runtime Escapes* — <https://doi.org/10.2172/1890908>
- **[S30]** *Escaping containers using the Dirty Pipe vulnerability* — <https://securitylabs.datadoghq.com/articles/dirty-pipe-container-escape-poc/>
- **[S31]** *An analysis of security vulnerabilities in container images for scientific data analysis* — <https://academic.oup.com/gigascience/article/10/6/giab025/6291571>
- **[S32]** *Containers do not contain* — <https://opensource.com/business/14/7/docker-security-selinux>
- **[S33]** *Kata Containers* — <https://katacontainers.io/>
- **[S34]** *OCI Runtime Specification — Virtual Machine Container Configuration* — <https://github.com/opencontainers/runtime-spec/blob/main/config-vm.md>
- **[S35]** *Isolation modes for Windows containers* — <https://learn.microsoft.com/en-us/virtualization/windowscontainers/manage-containers/hyperv-container>
- **[S36]** *Zero-Day Vulnerabilities in VMware ESXi* — <https://www.tenable.com/blog/cve-2025-22224-cve-2025-22225-cve-2025-22226-zero-day-vulnerabilities-in-vmware-esxi>
- **[S37]** *Containerization* — <https://github.com/apple/containerization>
- **[S38]** *Firecracker: Lightweight Virtualization for Serverless Applications* — <https://www.usenix.org/system/files/nsdi20-paper-agache.pdf>
- **[S39]** *Dockershim Removal FAQ* — <https://kubernetes.io/blog/2022/02/17/docker-shim-faq/>
- **[S40]** *Don't Panic: Kubernetes and Docker* — <https://kubernetes.io/blog/2020/12/02/dont-panic-kubernetes-and-docker/>
- **[S41]** *Memory inside Linux containers* — <https://fabiokung.com/2014/03/13/memory-inside-linux-containers/>
- **[S42]** *containers/common — pkg/config/default.go* — <https://github.com/containers/common/blob/main/pkg/config/default.go>
- **[S43]** *moby/moby — daemon/pkg/oci/caps/defaults.go* — <https://github.com/moby/moby/blob/master/daemon/pkg/oci/caps/defaults.go>
- **[S44]** *Docker on macOS is slow and how to fix it* — <https://www.cncf.io/blog/2023/02/02/docker-on-macos-is-slow-and-how-to-fix-it/>
- **[S45]** *Rosetta emulation on arm64 macOS breaks many amd64 images* — <https://github.com/docker/for-mac/issues/7137>

- **[S46]** Comparing WSL Versions — <https://learn.microsoft.com/en-us/windows/wsl/compare-versions>
- **[S47]** Systemd support in WSL — <https://learn.microsoft.com/en-us/windows/wsl/systemd>
- **[S48]** Ubuntu 24.04 LTS release notes — <https://documentation.ubuntu.com/release-notes/24.04/>
- **[S49]** Docker Hub usage and rate limits — <https://docs.docker.com/docker-hub/usage/>
- **[S50]** Docker Desktop license agreement — <https://docs.docker.com/subscription/desktop-license/>
- **[S51]** Install Docker Engine — <https://docs.docker.com/engine/install/>
- **[S52]** Podman Installation Instructions — <https://podman.io/docs/installation>
- **[S53]** Lima templates — <https://lima-vm.io/docs/templates/>
- **[S54]** Lima configuration — <https://lima-vm.io/docs/config/>
- **[S55]** podman-machine-init(1) — <https://docs.podman.io/en/latest/markdown/podman-machine-init.1.html>
- **[S56]** containers-from-scratch — <https://github.com/lizrice/containers-from-scratch>
- **[S57]** Predict-Observe-Explain — <https://science-education-research.com/teaching-science/constructivist-pedagogy/predict-observe-explain/>
- **[S58]** Solaris Zones: Operating System Support for Consolidating Commercial Workloads — https://www.usenix.org/legacy/event/lisa04/tech/full_papers/price/price.pdf
- **[S59]** Moving ARM64 Docker Builds Off QEMU — <https://www.warppbuild.com/guides/qemu-vs-native-arm64-builds>
- **[S60]** OCI Image Format Specification — Image Configuration — <https://github.com/opencontainers/image-spec/blob/main/config.md>
- **[S61]** podman-run(1) — Podman command reference — <https://docs.podman.io/en/latest/markdown/podman-run.1.html>
- **[S62]** An introduction to Docker for reproducible research — https://polaris.imag.fr/arnaud.legrand/research/readings/acm_sigops_si_rsea/p71-boettiger.pdf
- **[S63]** An Updated Performance Comparison of Virtual Machines and Linux Containers — <https://www.read.seas.harvard.edu/~kohler/class/cs260r-s19/container-perf14.pdf>
- **[S64]** Storage overview — Docker Docs — <https://docs.docker.com/engine/storage/>
- **[S65]** Podman v6.0.0 release notes — <https://github.com/podman-container-tools/podman/releases/tag/v6.0.0>

- **[S66]** *podman-compose(1)* — Podman manual page — <https://docs.podman.io/en/latest/markdown/podman-compose.1.html>
- **[S67]** *ptrace(2)* — Linux manual page — <https://man7.org/linux/man-pages/man2/ptrace.2.html>
- **[S68]** Resource constraints — Docker Docs — https://docs.docker.com/engine/containers/resource_constraints/
- **[S69]** *lxc/lxcfs* — project README — <https://github.com/lxc/lxcfs>
- **[S70]** containerd Runtime v2 (shim API) — <https://containerd.io/docs/2.3/runtime-v2/>
- **[S71]** Advanced settings configuration in WSL — <https://learn.microsoft.com/en-us/windows/wsl/wsl-config>
- **[S72]** Working across Windows and Linux file systems (WSL) — <https://learn.microsoft.com/en-us/windows/wsl/filesystems>
- **[S73]** Multi-tenancy — Kubernetes documentation — <https://kubernetes.io/docs/concepts/security/multi-tenancy/>
- **[S74]** Overlay Filesystem — the Linux Kernel documentation — <https://docs.kernel.org/filesystems/overlayfs.html>
- **[S75]** What is a container? — Docker Docs — <https://docs.docker.com/get-started/docker-concepts/the-basics/what-is-a-container/>
- **[S76]** Volumes — Docker Docs — <https://docs.docker.com/engine/storage/volumes/>
- **[S77]** Ubuntu 23.10 restricted unprivileged user namespaces — <https://ubuntu.com/blog/ubuntu-23-10-restricted-unprivileged-user-namespaces>
- **[S78]** *containers.conf(5)* — configuration reference — <https://github.com/containers/common/blob/main/docs/containers.conf.5.md>
- **[S79]** Troubleshooting Windows Subsystem for Linux — <https://learn.microsoft.com/en-us/windows/wsl/troubleshooting>
- **[S80]** Bind mounts — Docker Docs — <https://docs.docker.com/engine/storage/bind-mounts/>
- **[S81]** OCI Runtime Specification — Configuration — <https://github.com/opencontainers/runtime-spec/blob/main/config.md>
- **[S82]** Install Docker Engine on Ubuntu — <https://docs.docker.com/engine/install/ubuntu/>
- **[S83]** Homebrew: The Package Manager for Everywhere — <https://brew.sh>
- **[S84]** Yama — the Linux Kernel documentation — <https://docs.kernel.org/admin-guide/LSM/Yama.html>
- **[S85]** Linux post-installation steps for Docker Engine — <https://docs.docker.com/engine/install/linux-postinstall/>

- **[S86]** Homebrew — Installation — <https://docs.brew.sh/Installation>
- **[V1]** Containers with Docker and Podman — note in the author's research vault.
- **[V2]** Containers with Docker and Podman — note in the author's research vault.
- **[V3]** Containers with Docker and Podman — note in the author's research vault.
- **[V4]** Containers with Docker and Podman — note in the author's research vault.
- **[V5]** Containers with Docker and Podman — note in the author's research vault.
- **[V6]** Containers with Docker and Podman — note in the author's research vault.
- **[V7]** Containers with Docker and Podman — note in the author's research vault.
- **[V8]** Containers with Docker and Podman — note in the author's research vault.
- **[V9]** Containers with Docker and Podman — note in the author's research vault.
- **[V10]** Containers with Docker and Podman — note in the author's research vault.
- **[V11]** Containers with Docker and Podman — note in the author's research vault.
- **[V12]** Containers with Docker and Podman — note in the author's research vault.
- **[V13]** Containers with Docker and Podman — note in the author's research vault.
- **[V14]** Containers with Docker and Podman — note in the author's research vault.
- **[V15]** Containers with Docker and Podman — note in the author's research vault.
- **[V16]** Containers with Docker and Podman — note in the author's research vault.
- **[V17]** Containers with Docker and Podman — note in the author's research vault.
- **[V18]** Containers with Docker and Podman — note in the author's research vault.
- **[V19]** Copy-on-Write — note in the author's research vault.
- **[V20]** Copy-on-Write — note in the author's research vault.
- **[V21]** The System Call Boundary — note in the author's research vault.
- **[V22]** Where the Abstraction Leaks — note in the author's research vault.
- **[V23]** Where the Abstraction Leaks — note in the author's research vault.
- **[V24]** devcontainers — note in the author's research vault.
- **[V26]** Processes, Threads and Scheduling — note in the author's research vault.
- **[V27]** Containers with Docker and Podman — note in the author's research vault.
- **[V28]** Containers with Docker and Podman — note in the author's research vault.
- **[V29]** Containers with Docker and Podman — note in the author's research vault.
- **[V30]** Containers with Docker and Podman — note in the author's research vault.
- **[V31]** Containers with Docker and Podman — note in the author's research vault.
- **[V32]** Containers with Docker and Podman — note in the author's research vault.
- **[V33]** Containers with Docker and Podman — note in the author's research vault.
- **[V34]** Containers with Docker and Podman — note in the author's research vault.

- **[V35]** *Containers with Docker and Podman — note in the author's research vault.*
- **[V36]** *Containers with Docker and Podman — note in the author's research vault.*
- **[V37]** *Containers with Docker and Podman — note in the author's research vault.*
- **[V38]** *Containers with Docker and Podman — note in the author's research vault.*

The references document (PDF and web page) carries the full source list, and a summary of every [V] source with its own external references.

Source list only.